

Lexeme, Token, and Stream

Lexeme:

- **Definition:** A sequence of characters in the source code that matches a pattern for a token and is identified by the lexical analyzer as an instance of that token.
- **Example:** In the statement `int age = 25;`, the lexemes are: `int`, `age`, `=`, `25`, `;`.

Token:

- **Definition:** A pair consisting of a token name (or type) and an optional token value. The token name represents the category of a lexical unit, while the token value (if present) is the actual lexeme itself.
- **Example:**
 - `int`: Token name (keyword)
 - `age`: Token name (identifier), Token value: `"age"`
 - `=`: Token name (operator)
 - `25`: Token name (integer literal), Token value: `"25"`
 - `;`: Token name (semicolon)

Stream:

- **Definition:** A sequence of tokens generated by the lexical analysis phase of the compilation process. The stream is then fed into the parser for further processing.
- **Example:**
 - `<keyword, "int">`
 - `<identifier, "age">`
 - `<operator, "=">`
 - `<integer_literal, "25">`
 - `<semicolon, ";">`

In Summary:

- **Lexeme:** The actual sequence of characters in the source code.
- **Token:** An abstract representation of a lexeme, consisting of a token name and an optional value.
- **Stream:** A sequence of tokens that is passed to the parser for further processing.

Relationship:

1. **Lexemes are extracted from the source code.**
2. **Lexemes are classified and mapped into tokens.**
3. **Tokens are organized into a stream.**

Visual Representation:

Source Code: `int age = 25;`

Lexemes: `int`, `age`, `=`, `25`, `;`

Tokens:

```
<keyword, "int">
<identifier, "age">
<operator, "=">
<integer_literal, "25">
<semicolon, ";">
```

Token Stream: `<keyword, "int"> <identifier, "age"> <operator, "="> <integer_literal, "25"> <semicolon, ";">`